2023 2nd International Conference for Innovation in Technology (INOCON)
Bangalore, India. Mar 3-5, 2023

2023 2nd International Conference for Innovation in Technology (INOCON) | 979-8-3503-2092-3/23/$31.00 ©2023 IEEE | DOI: 10.1109/INOCON57975.2023.10101124

Body Posture Correction and Hand Gesture Detection Using Federated Learning and Mediapipe

Rijul Singhal[a], Hardik Modi[a], S Srihari[a], Advit Gandhi[a], Prakash C O[a], Sivaraman Eswaran[b]
[a] Dept. of Computer Science and Engineering, PES University, Bengaluru, India
[b]Dept. of Electrical and Computer Engineering, Curtin University Malasyia, Miri, 98009, Sarawak, Malasyia
rijulsinghal63@gmail.com, hdmodi738@gmail.com, shari250401@gmail.com, advitgandhi12@gmail.com,
coprakasha@pes.edu, sivaraman.eswaran@gmail.com

***Abstract*— In today's post-covid culture, where everyone works from home, there is a huge possibility of serious long-term health problems. A lot of people have started taking up exercises at home and if done incorrectly, they can have major negative effects. Another one of the main contributors to these health issues is bad sitting posture, which is only exacerbated when working for hours on end. Hand gesture detection has many useful applications in elderly healthcare, automating actions and gesture-based presentations and games. To help users with these actions, our paper proposes pinpointing the points of the error to the user in real-time and in a lightweight manner for yoga posture correction. The incorrect positions shall be shown in real-time on top of the user's video feed to help them correct it properly. The user shall be told about when they are sitting in a bad position, and the overall bad posture time will also be shown for the session, which will provide the required information to the user. To further help users in a useful manner, our paper looks to augment the hand gesture detection feature with federated learning and personalization to avoid the common pitfall of privacy concerns, while still allowing users to customize their experience. The proposed library for the implementation of these tasks is the MediaPipe library. This library is one of the key components that makes the features lightweight and easy to use. The aforementioned library also looks to implement the features in real time with no lag while keeping the resource requirements as low as possible.**

***Index Terms*—Federated Learning, MediaPipe, Gesture Detection, Body Posture Correction, Sitting Posture Correction**

I. Introduction

Many industries, including physical training and geriatric healthcare, use gesture detection. However, this has the drawback of requiring a lot of memory and storage due to the size of the training and testing data. Along with privacy issues, sharing to a centralized store also requires a lot of bandwidth.

A. Federated Learning

Federated Learning enables local systems to collaboratively learn a shared prediction model while keeping all the training data on the device, decoupling the ability to do machine learning from the need to store the data in the cloud[1]. This goes beyond the use of local models that make predictions on local systems by bringing model training to the device as well.

B. MediaPipe

MediaPipe is a framework that enables developers to build multi-modal (video, audio, time series data) cross-platform applied ML pipelines. 2 major features within MediaPipe are used here Mediapipe Pose and Mediapipe Hands

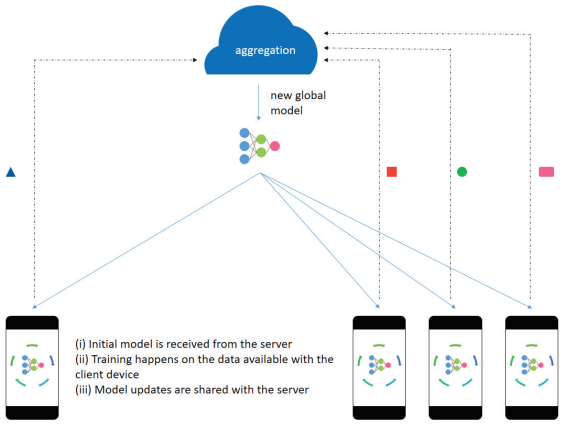

Fig. 1. Pictorial Representation of Federated Learning [1].

1) MediaPipe Pose: MediaPipe Pose converts the image to a set of 33 coordinates with the x, y, and z positions along with the visibility for the object in focus in each frame. These are the major joints of the human body on both the left and right sides, like the shoulder, hip, and knee. These coordinates are then used by the yoga posture correction and sitting posture correction features [2].

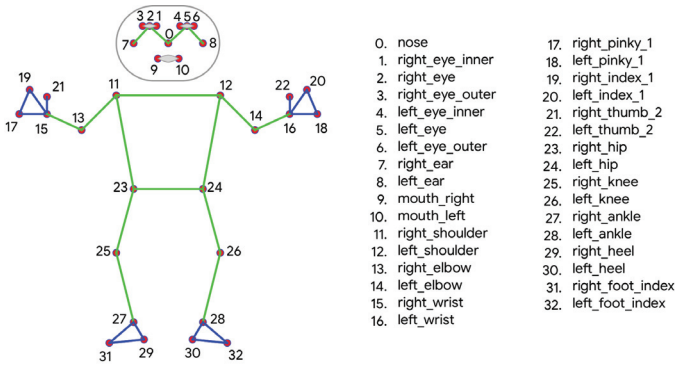

Fig. 2. Landmark Coordinates for each frame obtained from MediaPipe Pose [3].

2) MediaPipe Hands: MediaPipe Hands converts the image of a hand to a set of 21 landmark points, with their x, y, and z coordinates. Each finger has 4 landmark points, and there is another one at the base of the hand. This set of coordinates is used by the hand gesture detection feature [4].

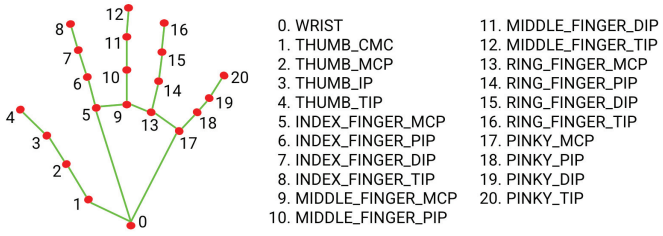

Fig. 3. Landmark Coordinates for each frame obtained from MediaPipe Hands [5].

C. Flower API

Federated learning is implemented here using the Flower framework. It is easy to integrate with any ML model and does not need any specific requirements for the model to work. It is implemented for the model in the hand gesture detection feature.

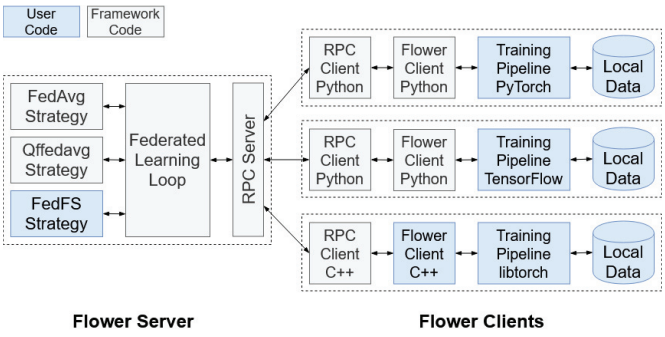

Fig. 4. Flower API Workflow [6].

II. Related Works

The above combination of requirements is where there is no substantial work done. Many works tackle parts of the problem individually, and building upon and integrating them is a key aspect to be looked into.

One such related work was mentioned in [7], where the game of Pong is personalized based on the skill level of the player. The main feature here is the FRL architecture provided by the authors. Several works utilize MediaPipe for related applications like sign detection and yoga posture classification like [8]. All of these provide a solid foundation on how to utilize MediaPipe and build upon it for the specific requirements needed here. Another related work is [9], which looks at privacy preservation in federated learning extensively. It looks at the threats and attack vectors in federated learning frameworks, and provides solutions to some of them. There are also works that implement hand gesture detection using many different algorithms. One such work which uses the Camshift algorithm and Haar-like features are [10]. A suitable comparison is needed to find the most appropriate one for our work and improve on it suitably.

To summarize, the prerequisites for this particular work are an understanding of gesture detection, how it is implemented, along with federated learning and how to integrate it with MediaPipe. The related works provide good starting points for each feature individually, and we must now look to bring them all together, and ideally improve them. This work can also be used and built upon further in the future as it has a lot of different applications and uses.

III. Proposed Methodology

The various approaches discussed in the previous section have tried to find the right balance between accuracy and computational costs. The process of conversion of images to coordinates is usually quite resource-intensive [11] [12]. Weighing the pros and cons, this research looks to use the MediaPipe library, which provides a lightweight solution to this issue and keeps up with the necessary accuracy threshold.

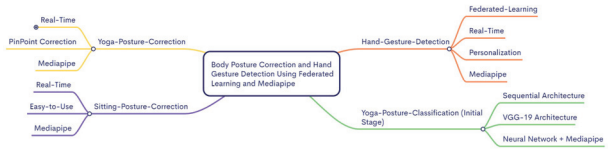

Fig. 5. Proposed Features for our Work.

A. Yoga Posture Correction

1) Develop the Correct Validation Parameters: The first step is to design and develop the validation parameters for each Yoga Posture based on the angles formed by the joints. To make this feature more general, we only use the angles between the required points. The angles provide a way to see the positions relative to each other, eliminating any effect the physical features of the user like BMI or height may have.

2) Capturing the Image: The next step involves capturing the image of the user performing the asana using the user's device. This can be done using a variety of methods and the one that integrates the best with the other elements is chosen.

3) Converting Image to Coordinates: The image captured is then converted to coordinates to be used to identify errors. The conversion of image points to a list of x, y, z, and visibility coordinates is done using the MediaPipe library.

4) Use Coordinates to Detect Errors: The coordinates obtained are then used to check certain important angles formed. The angles between certain joints are what make the yoga posture correct or incorrect. If the angles are all within a suitable threshold, there is no error. Else, the incorrect positions need to be mentioned and displayed.

5) Display Errors: Finally, the users' correct points are highlighted in green while the incorrect ones are highlighted in red.

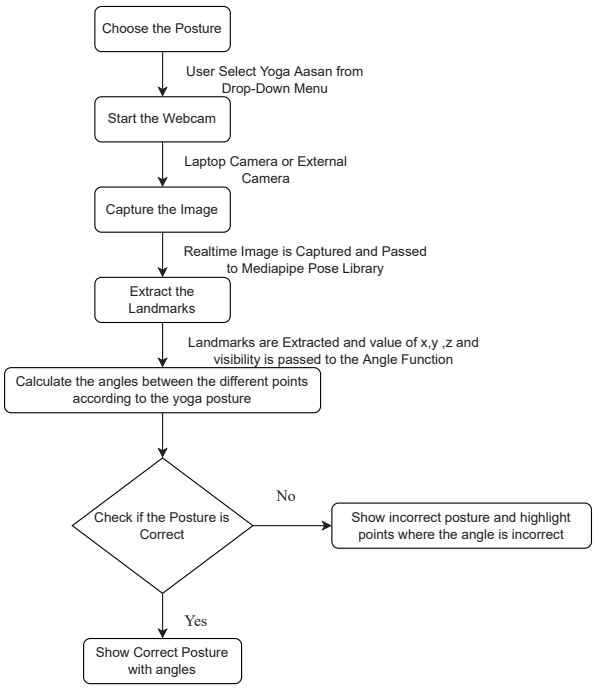

Fig. 6. Work-Flow Diagram of Yoga Posture Correction.

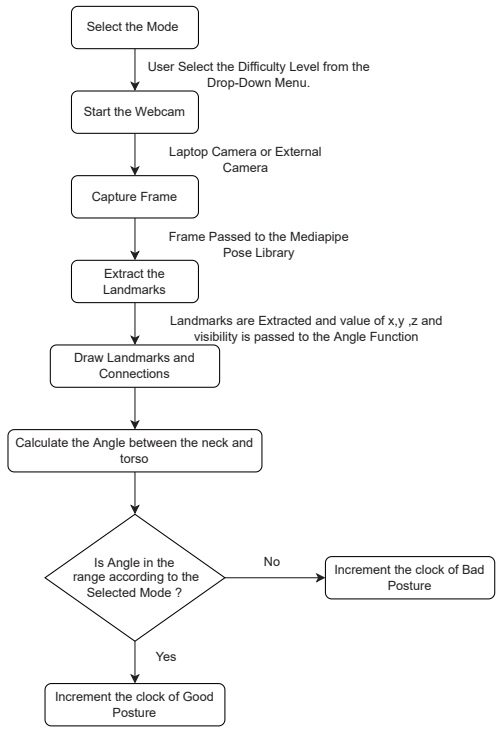

Fig. 7. Work-Flow Diagram of Sitting Posture Correction.

B. Sitting Posture Correction

1) Capture the video frame/image: The first step is to capture the video frame of the user while they are sitting and working on their computer, using the laptops or an external camera. Subsequently, the video frames captured by the camera are converted into a sequence of images, which are used for further processing.

2) Extracting coordinates from images: The x,y,z, and visibility coordinates are extracted using the MediaPipe library. The coordinates of the 33 landmark points of the human body are then used for error detection in the posture of the user, while they are sitting.

3) Use the coordinates to detect errors: After extracting the landmarks and coordinates which consist of x,y,z, and visibility, from the MediaPipe library, we use the y and z coordinates between the neck and the torso to determine the angle between the back and the neck. This is to determine whether the user is maintaining a straight and correct posture, or if the user is slouching. We have chosen this approach because it eliminates the need for any additional cameras, or a specific position for the camera (usually on the side), which makes this feature user-friendly.

4) Display Errors: After the determination of posture as correct or incorrect, the appropriate feedback is displayed to the user on the screen.

C. Hand Gesture Detection

The user has to make sure that their hand is completely visible in the camera.

1) Capturing the Video Frame/Image: The first step is to capture the video frame of the user's hands while the user using the Hand Gesture Detection Application. Subsequently, the video frames captured by the camera are converted into a sequence of images [13], which are used for further processing.

2) Extracting coordinates from images: The x,y,z, and visibility coordinates are extracted using MediaPipe Hands. The coordinates of the 21 landmark points of the human hand are then used to detect the specific hand gesture.

3) Normalizing the Landmarks: To normalize the output received from MediaPipe, we are finding the distance of all the landmarks shown in the image above. From the landmark point 0, and then depending on the maximum and minimum distance of the landmark point from landmark 0, the distances are normalized to values between (-1, 1). This is done to allow the hand to be placed anywhere within the frame, but give the same level of confidence in the detection and classification of the hand gesture.

4) Storing in CSV format: The normalized coordinates are stored in a CSV file so that they can be fed into the neural

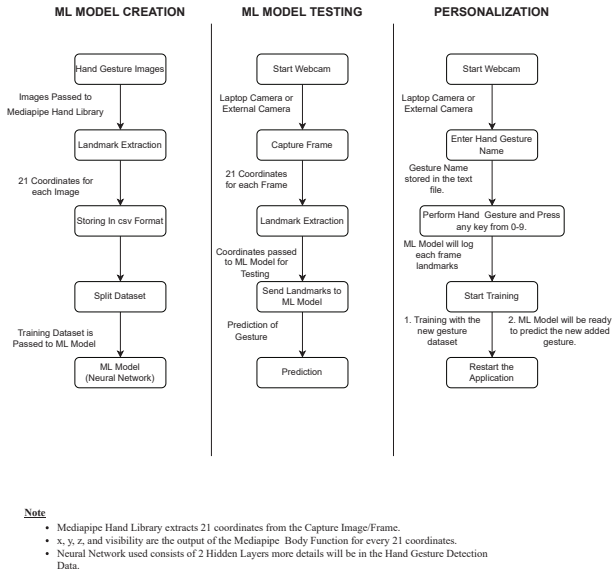

Fig. 8. Work Flow Diagram of Hand Gesture Detection.

network and learned from.

5) Using Neural Network to classify: The 21 normalized landmarks of the hands stored in a CSV format is fed into the neural network for the classification of the gestures.

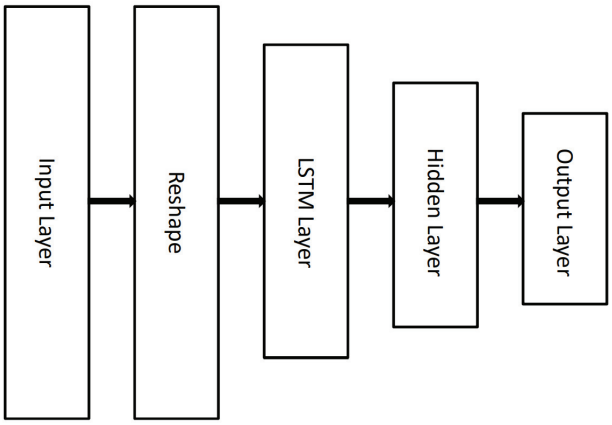

Fig. 9. Neural Network Architecture (Moving Gestures).

6) Federating the model: We have used the Flower federated learning framework to federate the model. The Flower framework is a very user-friendly framework that allows us to easily specify all the required configurations to implement federated learning. These include:

- Minimum number of clients that should be up before FL can start.
- Minimum number of fit clients that should return updated weights after training if the central server has started a new round of train.
- The Federated Learning algorithm should be used.

We have essentially implemented federating learning for the pre-defined gestures in the dataset. If the accuracy of the local model increases beyond that of the central model, then the weights are updated to the central model.

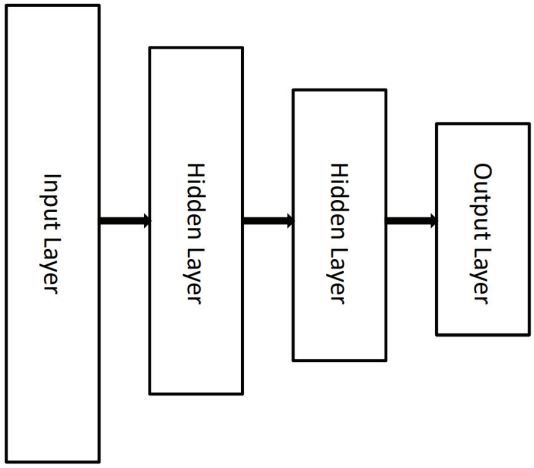

Fig. 10. Neural Network Architecture (Static Gestures).

7) Personalization: Finally, we have also implemented a personalization feature, wherein the client can add gestures of their liking to their local device. This leads to the personalization of the federated learning model. The new gestures added by the client are stored locally and are not sent to the server. This is done to help in personalizing each user's experience to their liking.

IV. Results and Discussion

The results obtained from the implementation of the different features are compiled and discussed here.

A. Yoga Posture Correction

The calculation of the angles of the joints is done using the relative positions of the three required landmark points. The angle thus obtained is free of any influence from the size or body-mass index(BMI) of the user performing it. So, there is no tuning to be done based on the user's physical parameters, and it works regardless of any differences between users. The correction works in real time and has no lag.

Fig. 11. Working of Yoga Posture Correction.

In total, 13 yoga postures are implemented and the application is lightweight. The correction parameters are shown on necessary joints so that the user can easily navigate to the areas that need to be corrected.

B. Sitting Posture Correction

Sitting posture detection has been implemented with a fully functional front-end, and we have implemented three different modes within sitting posture detection - lenient, medium, and strict. In the front end, the user is displayed a "good posture time" and a "bad posture time" in the form of a timer, depending on the user's posture.

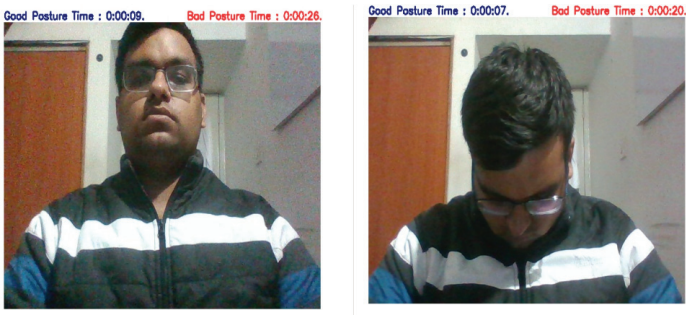

Fig. 12. Working of Sitting Posture Correction.

In lenient mode, we are only considering the angle of the neck, and whether the person's neck is straight or not.

In medium mode, both the neck and the back are considered, and the posture is classified as good or bad depending on whether the neck and back are both relatively straight or not.

Finally, strict mode, which is a stricter version of the medium mode, considers both the neck and the back, and the person has to sit straight for the posture to be classified as a good posture.

C. Hand Gesture Detection

Gesture detection also has a fully functional front-end, and it has been implemented while incorporating federated learning. It offers real-time classification of the gestures without any lag on any average modern computer. Our gesture classification model has achieved an accuracy of 94.08% for classification among the 10 pre-defined gestures.

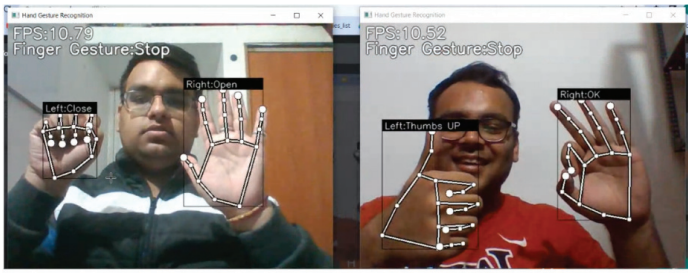

Fig. 13. Gesture Detection using Federated Learning, with two clients running simultaneously.

There is also a personalization aspect implemented in the gesture detection model, wherein a particular user can add their gestures into the model. However, the accuracy decreases as more gestures are added and the classes increase, and the accuracy achieved is also a function of the number of training examples available. Thus, if a significant number of training examples are added for the newly added gesture, the accuracy will be improved.

D. Yoga Posture Classification

The yoga posture feature is extremely useful to anyone who practices yoga or wants to be able to find and correct any errors in their postures. This is very helpful for both professionals and new learners and is very intuitive. Here, we manually have to select the yoga postures that need correction. This can be extended by automating the recognition process. It can be solved using proper posture classification techniques. The results of the initial stages of work for this part are as mentioned below:

1) Sequential Architecture: The training and validation accuracy can be observed in the below figure. It is clear to see that the validation accuracy is close to the training accuracy till a certain point and then falls off. The figure on the right shows the training and validation loss for the same. The same can also be observed there.

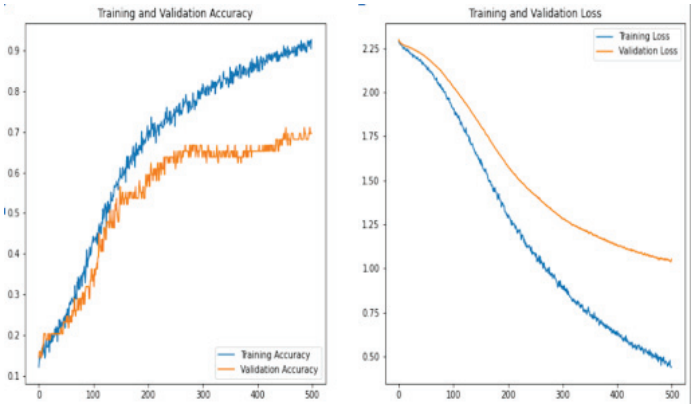

Fig. 14. Sequential Architecture

2) VGG-19 Architecture: The results obtained for this architecture and activation function are hugely different. There is a clear difference in training and validation accuracy initially but once they meet, they are very close. Overall, the response time for the former is better than the latter. However, the overall validation accuracy is generally higher in the latter case, and the loss is lower. The first architecture is unable to run in real-time, and there is a lag of 4-5 seconds noticed when the VGG-19 architecture is used.

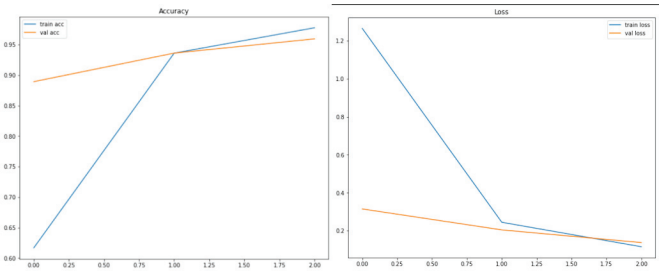

Fig. 15. VGG-19 Architecture

3) MediaPipe and Neural Network: When using MediaPipe with a basic neural network, the application runs in real-time with no lag, but the accuracy is relatively low (around 70%). The solution to this issue may be to use MediaPipe along with a more complex neural network. Adding more layers to the neural network should ideally deal with both of these issues.

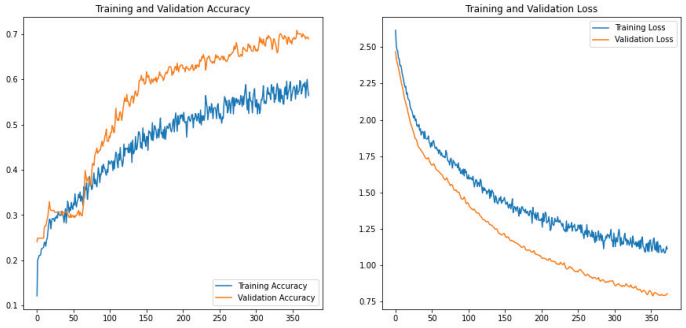

Fig. 16. MediaPipe and Neural Network

V. Conclusion

The Sitting Posture Correction feature works in real-time and is lightweight as it does not require any Machine Learning model. The front view of the user is good enough to correct the posture, and thus the user can continue working without any extra devices and additional hassles.

The Yoga Posture Correction feature is similar to the Sitting Posture Correction feature in working in real-time and being lightweight as it does not require any Machine Learning model. This feature also pinpoints the exact locations at which the corrections are necessary.

Gesture Detection has been implemented while incorporating Federated Learning, with an accuracy of more than 90% based on the number of gestures to be classified. Personalization is also added as a feature, wherein the user can add their gestures on their local machine.

VI. Future Work

The Yoga Posture Correction feature requires yoga postures to be selected manually for correction. This process can be automated by using Mediapipe along with a neural network. Due to this reason, we have also included the initial stages of Yoga Posture Classification work in the above sections. Future works on this topic could look to use a more complex neural network with more layers along with MediaPipe to potentially solve this issue.

One possible vector to extend the work done here is to generalize the yoga posture correction feature to more common exercise types like daily workouts and stretches. This will require specific and in-depth knowledge of the related fields to be done properly.

Another possible use for this work might be to use it in other related works as a basic foundation for something more complex. The work done here should be able to provide a good starting point for other works to build upon.

References

[1] K. Chandorikar, "Introduction to federated learning and privacy preservation," 2019, [Online; accessed January 5, 2023].

[2] A. K. Singh, V. A. Kumbhare, and K. Arthi, "Real-time human pose detection and recognition using mediapipe," *International Conference on Soft Computing and Signal Processing*, 2022.

[3] Mediapipe Dev, "Pose tracking full body landmarks," 2019, [Online; accessed January 5, 2023].

[4] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann, "Mediapipe hands: On-device real-time hand tracking," *Google Research*, 2020.

[5] Mediapipe Dev, "Hand landmarks," 2019, [Online; accessed January 5, 2023].

[6] Flower Dev, "Flower pipeline," 2021, [Online; accessed January 5, 2023].

[7] C. Nadiger, A. Kumar, and S. Abdelhak, "Federated reinforcement learning for fast personalization," in *2019 IEEE Second International Conference on Artificial Intelligence and Knowledge Engineering (AIKE)*, 2019, pp. 123–127.

[8] A. Haldera and A. Tayade, "Real-time vernacular sign language recognition using mediapipe and machine learning," *International Journal of Research Publication and Reviews*, 2021.

[9] N. Truong, K. Sun, S. Wang, F. Guitton, and Y. Guo, "Privacy preservation in federated learning: An insightful survey from the gdpr perspective," *Computers I& Security*, vol. 110, p. 102402, 2021.

[10] Y. Yu, S. Bi, Y. Mo, and W. Qiu, "Real-time gesture recognition system based on camshift algorithm and haar-like feature," in *2016 IEEE International Conference on Cyber Technology in Automation, Control, and Intelligent Systems (CYBER)*, 2016, pp. 337–342.

[11] J. Zamora-Mora and M. Chacón-Rivas, "Real-time hand detection using convolutional neural networks for costa rican sign language recognition," in *2019 International Conference on Inclusive Technologies and Education (CONTIE)*, 2019, pp. 180–1806.

[12] L. Xie and X. Guo, "Object detection and analysis of human body postures based on tensorflow," *IEEE*, 2019.

[13] T. Sharp, C. Keskin, D. Robertson, J. Taylor, J. Shotton, D. Kim, C. Rhemann, I. Leichter, A. Vinnikov, Y. Wei, D. Freedman, P. Kohli, E. Krupka, A. Fitzgibbon, and S. Izadi, "Accurate, robust, and flexible real-time hand tracking," *Microsoft Research*, 2015.